

Detecting a Power Outage from Space

EAGLE-I outage reports + NASA Black Marble nighttime lights, Oahu, January 2025

```
library(tidyverse)
library(tigris)
library(sf)
library(blackmarbler)
library(purrr)
library(terra)
library(tidyterra)
library(lubridate)
library(arrow)
library(here)
library(exactextractr)
library(tmap)
library(mapview)
library(gt)

options(tigris_use_cache = TRUE)
```

NASA Earthdata credentials are needed to pull Black Marble data. Store these as environment variables rather than typing them directly into a script.

```
bearer <- get_nasa_token(username = "connorflynn",
                        password = "EarthData120")
```

Finding the right night to look at

VIIRS, the satellite behind Black Marble, passes over Hawaii at roughly 1-2 AM local time. Rather than picking whichever night had the worst outages overall, we look specifically at what EAGLE-I reported during that 1-2 AM window, since that's the moment a Black Marble image for a given date would actually be capturing.

```
eagle_i_hawaii <- read_feather(here("data/hawaii_outages_2025.feather")) |>
  filter(county != "Kalawao")

message("run_start_time timezone: ", attr(eagle_i_hawaii$run_start_time, "tzone"))
```

run_start_time timezone: UTC

`run_start_time` is stored in UTC. Hawaii is UTC-10 with no daylight saving, so the local overpass window has to be found in HST, not by filtering UTC hours directly.

```
eagle_i_hawaii <- eagle_i_hawaii |>
  mutate(run_start_time_hst = with_tz(run_start_time, tzone = "Pacific/Honolulu"))

overpass_window <- eagle_i_hawaii |>
  mutate(
    hour = hour(run_start_time_hst),
    date = as.Date(run_start_time_hst)
  ) |>
  filter(hour %in% c(1, 2))
```

Each date is scored on how large and how widespread the outage was, specifically within that overpass window.

```
overpass_summary <- overpass_window |>
  group_by(date) |>
  summarize(
    mean_customers_out = mean(customers_out, na.rm = TRUE),
    max_customers_out = max(customers_out, na.rm = TRUE),
    n_counties_affected = n_distinct(county[customers_out > 0]),
    .groups = "drop"
  ) |>
  mutate(
    score = scales::rescale(max_customers_out) +
      scales::rescale(n_counties_affected)
  ) |>
  arrange(desc(score))
```

```
print(overpass_summary)
```

```
# A tibble: 365 × 5
```

	date <date>	mean_customers_out <dbl>	max_customers_out <int>	n_counties_affected <int>	score <dbl>
1	2025-01-31	5349.	19537	4	2
2	2025-07-20	722.	2132	4	1.11
3	2025-08-03	677.	1971	4	1.10
4	2025-03-04	277.	1643	4	1.08
5	2025-11-07	287.	1592	4	1.08
6	2025-04-19	357.	1148	4	1.06
7	2025-11-13	375.	984	4	1.05
8	2025-02-01	235.	878	4	1.04
9	2025-05-17	153.	502	4	1.03
10	2025-11-20	86	490	4	1.02

```
# i 355 more rows
```

```
storm_date_label <- overpass_summary$date[1]  
message("Selected storm date (by overpass-window severity): ", storm_date_label)
```

```
Selected storm date (by overpass-window severity): 2025-01-31
```

```
overpass_window |>  
  filter(date == storm_date_label) |>  
  summarize(  
    mean_customers_out_at_overpass = mean(customers_out, na.rm = TRUE),  
    max_customers_out_at_overpass = max(customers_out, na.rm = TRUE),  
    n_counties_affected_at_overpass = n_distinct(county[customers_out > 0])  
  )
```

	mean_customers_out_at_overpass	max_customers_out_at_overpass
1	5349.188	19537

	n_counties_affected_at_overpass
1	4

The chart below shows the full outage timeline around the selected date, with the actual 1-2 AM overpass window shaded rather than marked with a single instantaneous line, since the exact overpass moment within that window isn't known precisely.

```
overpass_start <- as.POSIXct(paste(storm_date_label, "01:00:00"),
                              tz = "Pacific/Honolulu")
overpass_end <- as.POSIXct(paste(storm_date_label, "02:59:59"),
                            tz = "Pacific/Honolulu")

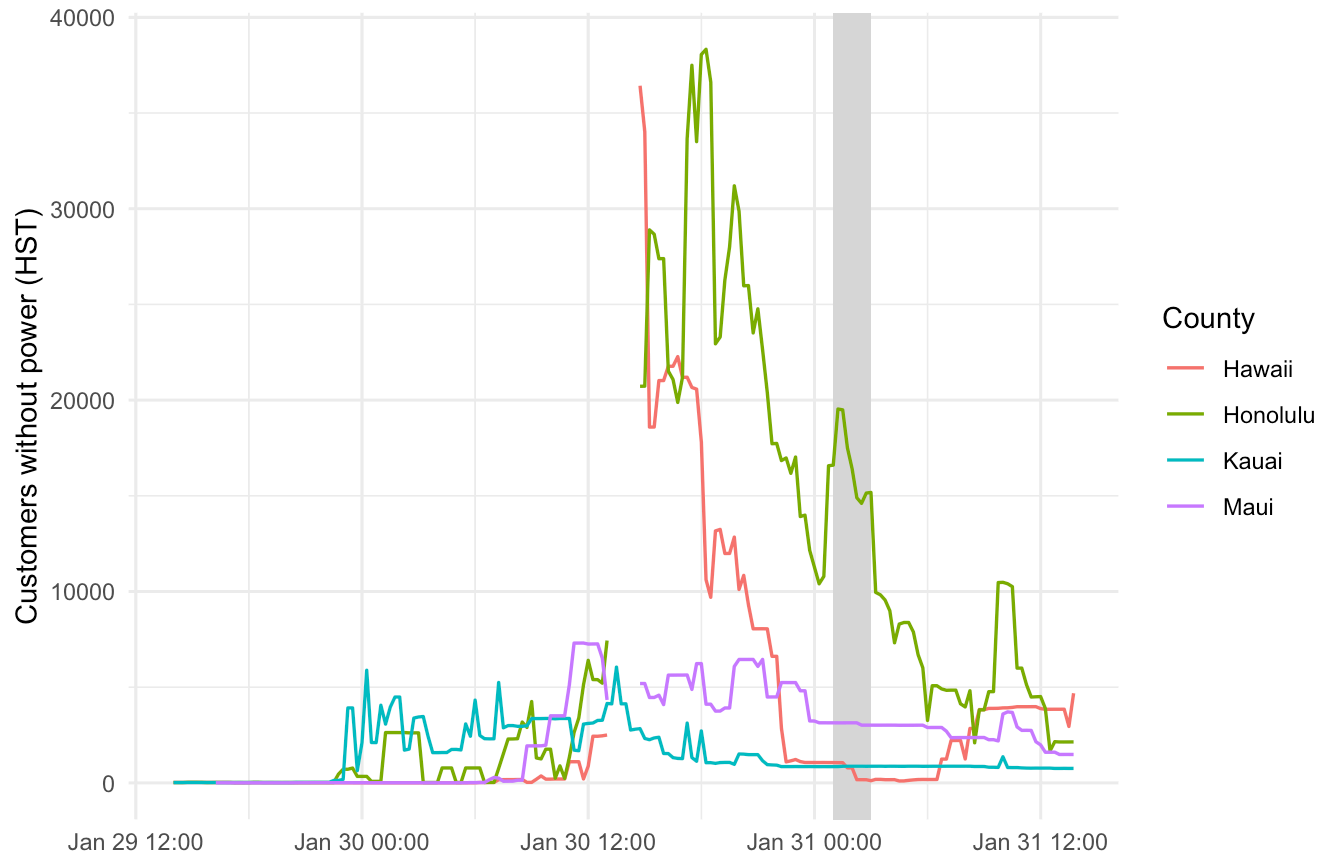
eagle_i_hawaii |>
  filter(as.Date(run_start_time_hst) %in% (storm_date_label + c(-1, 0))) |>
  ggplot(aes(x = run_start_time_hst, y = customers_out, color = county)) +
  geom_rect(aes(xmin = overpass_start, xmax = overpass_end, ymin = -Inf, ymax = Inf),
            inherit.aes = FALSE, fill = "grey85", alpha = 0.5) +
  geom_line(linewidth = 0.6) +
  labs(title = paste0("Outage Timeline – VIIRS Overpass Window Shaded (", storm_date_label, ")"),
       subtitle = "Shaded band shows the ~1-2 AM HST window Black Marble would have actually captured",
       x = NULL, y = "Customers without power (HST)", color = "County") +
  theme_minimal()
```

Warning in geom_rect(aes(xmin = overpass_start, xmax = overpass_end, ymin = -Inf, : All aesthetics have length 1, but the data has 727 rows.

! Please consider using `annotate()` or provide this layer with data containing a single row.

Outage Timeline — VIIRS Overpass Window Shaded (2025-01-31)

Shaded band shows the ~1-2 AM HST window Black Marble would have actually captured



Setting the baseline and storm dates

The baseline is the 10 days immediately preceding the storm. This window happens to overlap Kilauea's summit eruption, which entered its fourth eruptive episode on January 15, 2025 and was still actively fountaining as of the January 18 HVO report. That's kept in the baseline intentionally rather than avoided, since its effect on Hawaii County shows up clearly in the county-level chart further down.

```
storm_dates <- storm_date_label  
baseline_dates <- seq.Date(from = as.Date("2025-01-15"), to = as.Date("2025-01-24"), by = "day")
```

```
all_dates <- c(baseline_dates, storm_dates)
```

Hawaii boundary

`tigris` defaults to NAD83; Black Marble tiles expect WGS84, so a reprojection is required. The boundary is also cropped to the main islands, since Honolulu County's official boundary technically extends across the Northwestern Hawaiian Islands.

```
roi_hawaii <- counties(state = "HI", cb = TRUE, year = 2022) |>
  st_transform(4326) |>
  st_crop(xmin = -160.5, xmax = -154.5, ymin = 18.5, ymax = 22.5) |>
  filter(NAMELSAD != "Kalawao County") |>
  mutate(county = gsub(" County", "", NAME))
```

Warning: attribute variables are assumed to be spatially constant throughout all geometries

Pulling Black Marble rasters

Each date is pulled individually and wrapped in `tryCatch()`, so one missing or failed date doesn't stop the whole batch.

```
pull_one_date <- function(d) {
  tryCatch({
    bm_raster(roi_sf = roi_hawaii,
              product_id = "VNP46A2",
              date = d,
              bearer = bearer,
              quality_flag_rm = 2)
  }, error = function(e) {
    message("Failed for ", d, ": ", conditionMessage(e))
    NULL
  })
}
```

```
r_list <- map(all_dates, pull_one_date)
```

Warning in CPL_read_ogr(dsn, layer, query, as.character(options), quiet, : GDAL
Message 1: GDAL was built against curl 8.6.0, but is running against 7.77.0.
Runtime failure is likely !

Processing 3 nighttime light tiles

Processing: VNP46A2.A2025015.h01v06.002.2025047183351.h5

Processing: VNP46A2.A2025015.h02v06.002.2025047184321.h5

Processing: VNP46A2.A2025015.h02v07.002.2025047184328.h5

Processing 3 nighttime light tiles

Processing: VNP46A2.A2025016.h01v06.002.2025047184317.h5

Processing: VNP46A2.A2025016.h02v06.002.2025047185243.h5

Processing: VNP46A2.A2025016.h02v07.002.2025047185247.h5

Processing 3 nighttime light tiles

Processing: VNP46A2.A2025017.h01v06.002.2025047190145.h5

Processing: VNP46A2.A2025017.h02v06.002.2025047191111.h5

Processing: VNP46A2.A2025017.h02v07.002.2025047191118.h5

Processing 3 nighttime light tiles

Processing: VNP46A2.A2025018.h01v06.002.2025047192043.h5

Processing: VNP46A2.A2025018.h02v06.002.2025047193028.h5

Processing: VNP46A2.A2025018.h02v07.002.2025047193017.h5

Processing 3 nighttime light tiles

Processing: VNP46A2.A2025019.h01v06.002.2025047194055.h5

Processing: VNP46A2.A2025019.h02v06.002.2025047194943.h5

Processing: VNP46A2.A2025019.h02v07.002.2025047194950.h5

Processing 3 nighttime light tiles

Processing: VNP46A2.A2025020.h01v06.002.2025047195837.h5

Processing: VNP46A2.A2025020.h02v06.002.2025047200755.h5

Processing: VNP46A2.A2025020.h02v07.002.2025047200749.h5

Processing 3 nighttime light tiles

Processing: VNP46A2.A2025021.h01v06.002.2025047201644.h5

Processing: VNP46A2.A2025021.h02v06.002.2025047202610.h5

Processing: VNP46A2.A2025021.h02v07.002.2025047203512.h5

Processing 3 nighttime light tiles

Processing: VNP46A2.A2025022.h01v06.002.2025047204413.h5

Processing: VNP46A2.A2025022.h02v06.002.2025047205309.h5

Processing: VNP46A2.A2025022.h02v07.002.2025047210216.h5

Processing 3 nighttime light tiles

Processing: VNP46A2.A2025023.h01v06.002.2025047211039.h5

Processing: VNP46A2.A2025023.h02v06.002.2025047211941.h5

Processing: VNP46A2.A2025023.h02v07.002.2025047212848.h5

Processing 3 nighttime light tiles

Processing: VNP46A2.A2025024.h01v06.002.2025047213721.h5

Processing: VNP46A2.A2025024.h02v06.002.2025047213726.h5

Processing: VNP46A2.A2025024.h02v07.002.2025047214543.h5

Processing 3 nighttime light tiles

Processing: VNP46A2.A2025031.h01v06.002.2025047225656.h5

Processing: VNP46A2.A2025031.h02v06.002.2025047225710.h5

Processing: VNP46A2.A2025031.h02v07.002.2025047230538.h5

```
names(r_list) <- as.character(all_dates)
r_list <- compact(r_list)

message("Dates successfully pulled: ", paste(names(r_list), collapse = ", "))
```

Dates successfully pulled: 2025-01-15, 2025-01-16, 2025-01-17, 2025-01-18, 2025-01-19, 2025-01-20, 2025-01-21, 2025-01-22, 2025-01-23, 2025-01-24, 2025-01-31

```
message("Dates dropped (no imagery / failed): ",
       paste(setdiff(as.character(all_dates), names(r_list)), collapse = ", "))
```

Dates dropped (no imagery / failed):

```
r_all <- rast(r_list)

baseline_ok <- intersect(as.character(baseline_dates), names(r_all))
storm_ok <- intersect(as.character(storm_dates), names(r_all))

r_baseline <- r_all[[baseline_ok]]
r_storm <- r_all[[storm_ok]]
```

Masking and completeness

Everything from here on stays on the raw radiance scale. Log transforms are applied only when needed for a plot, and always fresh, never on top of an already-logged raster.

```
r_baseline <- terra::mask(r_baseline, roi_hawaii)
r_storm <- terra::mask(r_storm, roi_hawaii)

completeness_check <- tibble(
  date = c(baseline_ok, storm_ok),
  period = c(rep("Baseline", length(baseline_ok)), rep("Storm", length(storm_ok))),
  n_valid_pixels = c(
    sapply(seq_len(nlyr(r_baseline)), function(i) sum(!is.na(values(r_baseline[[i]])))),
    sapply(seq_len(nlyr(r_storm)), function(i) sum(!is.na(values(r_storm[[i]]))))
  )
)
print(completeness_check)
```

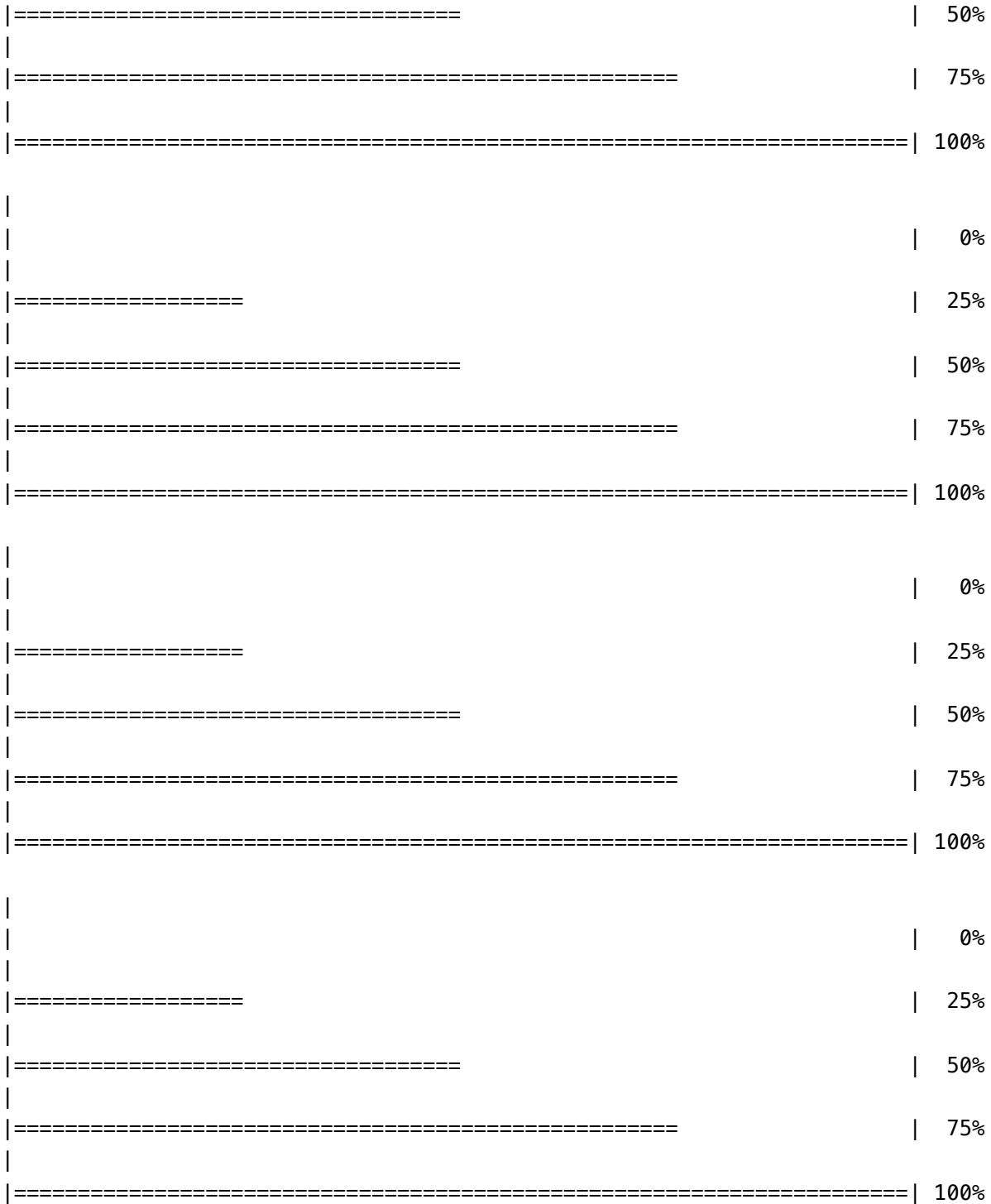
```
# A tibble: 11 × 3
  date      period  n_valid_pixels
  <chr>    <chr>         <int>
1 2025-01-15 Baseline      85207
2 2025-01-16 Baseline      85207
3 2025-01-17 Baseline      85207
4 2025-01-18 Baseline      85207
5 2025-01-19 Baseline      85207
6 2025-01-20 Baseline      85207
7 2025-01-21 Baseline      85207
8 2025-01-22 Baseline      85207
9 2025-01-23 Baseline      85207
10 2025-01-24 Baseline      85207
11 2025-01-31 Storm        85207
```

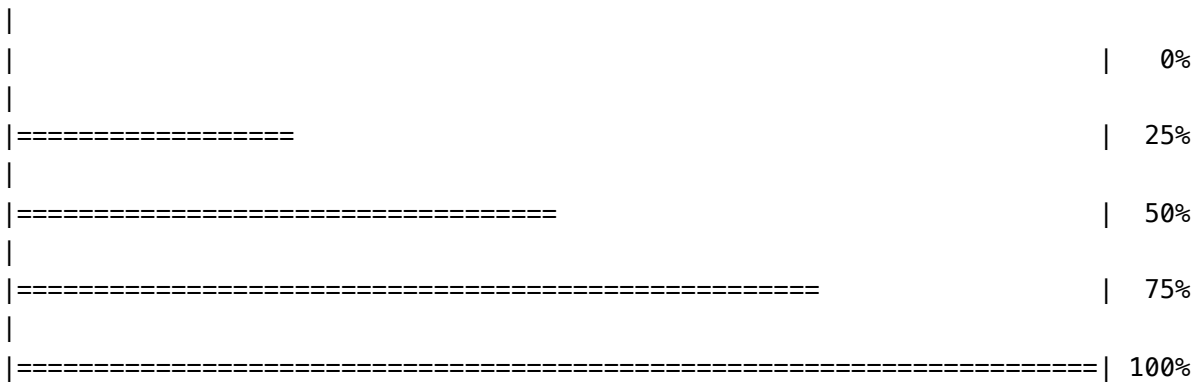
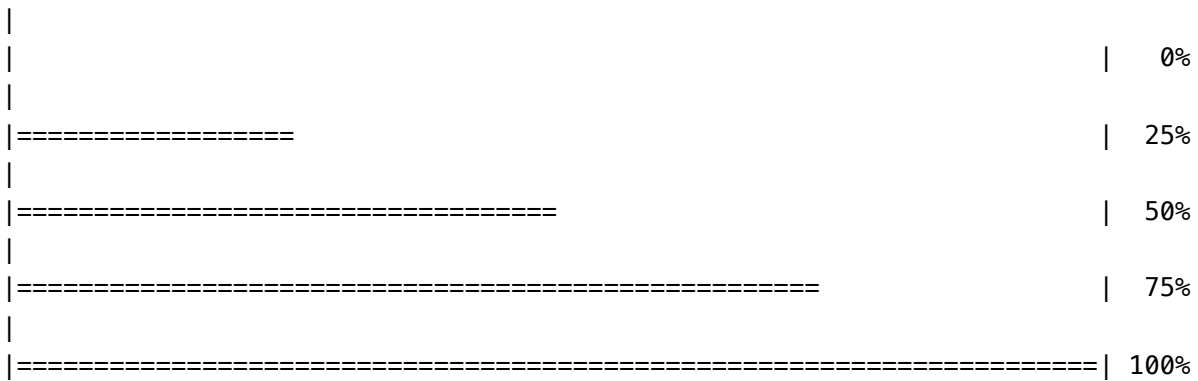
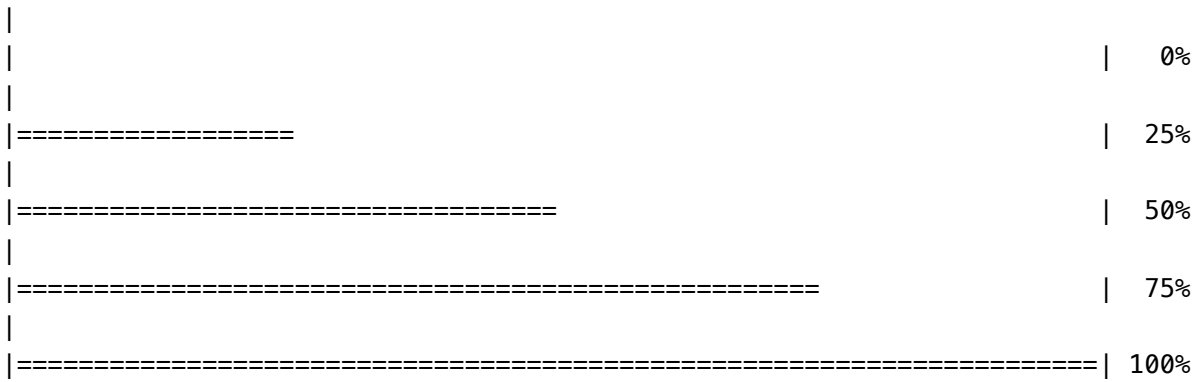
County-level luminance

County-level daily means are computed directly from the rasters already pulled above, rather than requesting the same tiles again through a separate summary function.

```
extract_from_local <- function(raster_stack, roi_sf) {  
  map_dfr(seq_len(nlyr(raster_stack)), function(i) {  
    layer <- raster_stack[[i]]  
    vals <- exact_extract(layer, roi_sf, fun = "mean")  
    tibble(  
      county = roi_sf$county,  
      date = as.Date(names(raster_stack)[i]),  
      ntl_mean = vals  
    )  
  })  
}  
  
ntl_trend <- bind_rows(  
  extract_from_local(r_baseline, roi_hawaii) |> mutate(period = "Baseline"),  
  extract_from_local(r_storm, roi_hawaii) |> mutate(period = "Storm")  
)
```

```
|  
|  
|  
|=====| 25%  
|  
|=====| 50%  
|  
|=====| 75%  
|  
|=====| 100%  
  
|  
|  
|  
|=====| 25%  
|
```







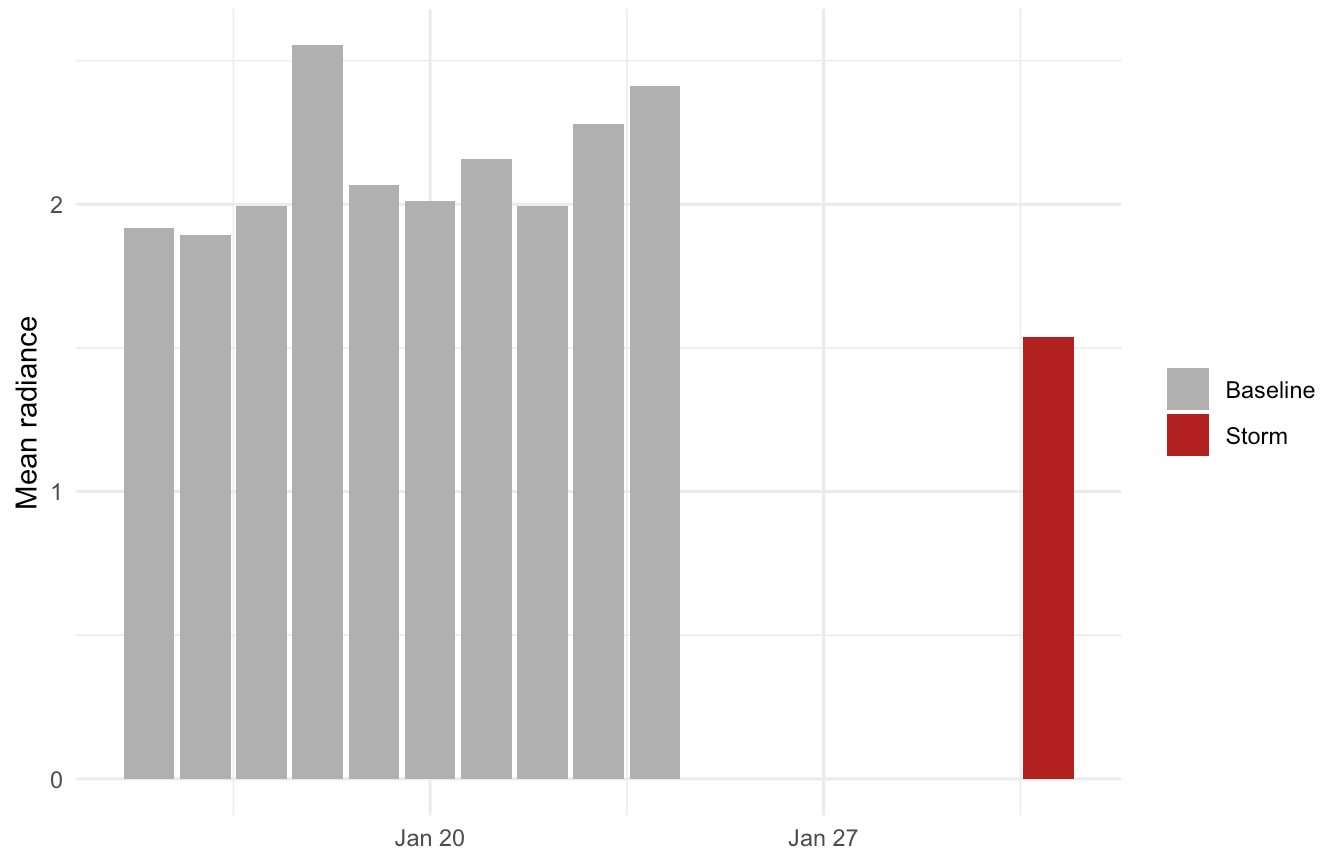
```
statewide_trend <- ntl_trend |>
  group_by(date, period) |>
  summarize(ntl_mean = mean(ntl_mean, na.rm = TRUE), .groups = "drop")

ggplot(statewide_trend, aes(x = date, y = ntl_mean, fill = period)) +
  geom_col() +
  scale_fill_manual(values = c("Baseline" = "grey70", "Storm" = "firebrick")) +
  labs(title = "Statewide Mean Nighttime Radiance by Day",
       subtitle = paste0("Baseline (Jan 15–24, 2025) vs. Storm Night (", storm_date_label, ")"),
```

```
x = NULL, y = "Mean radiance", fill = NULL) +  
theme_minimal()
```

Statewide Mean Nighttime Radiance by Day

Baseline (Jan 15-24, 2025) vs. Storm Night (2025-01-31)



Breaking this out by county shows Hawaii County's baseline sitting well above what a typical pre-storm night should look like — a real, independent light source (lava fountain glow) unrelated to the electrical grid.

```
kilauea_dates <- ntl_trend |>  
  filter(county == "Hawaii", period == "Baseline", date >= as.Date("2025-01-15")) |>  
  pull(date)  
  
ggplot(ntl_trend, aes(x = date, y = ntl_mean, fill = period)) +
```

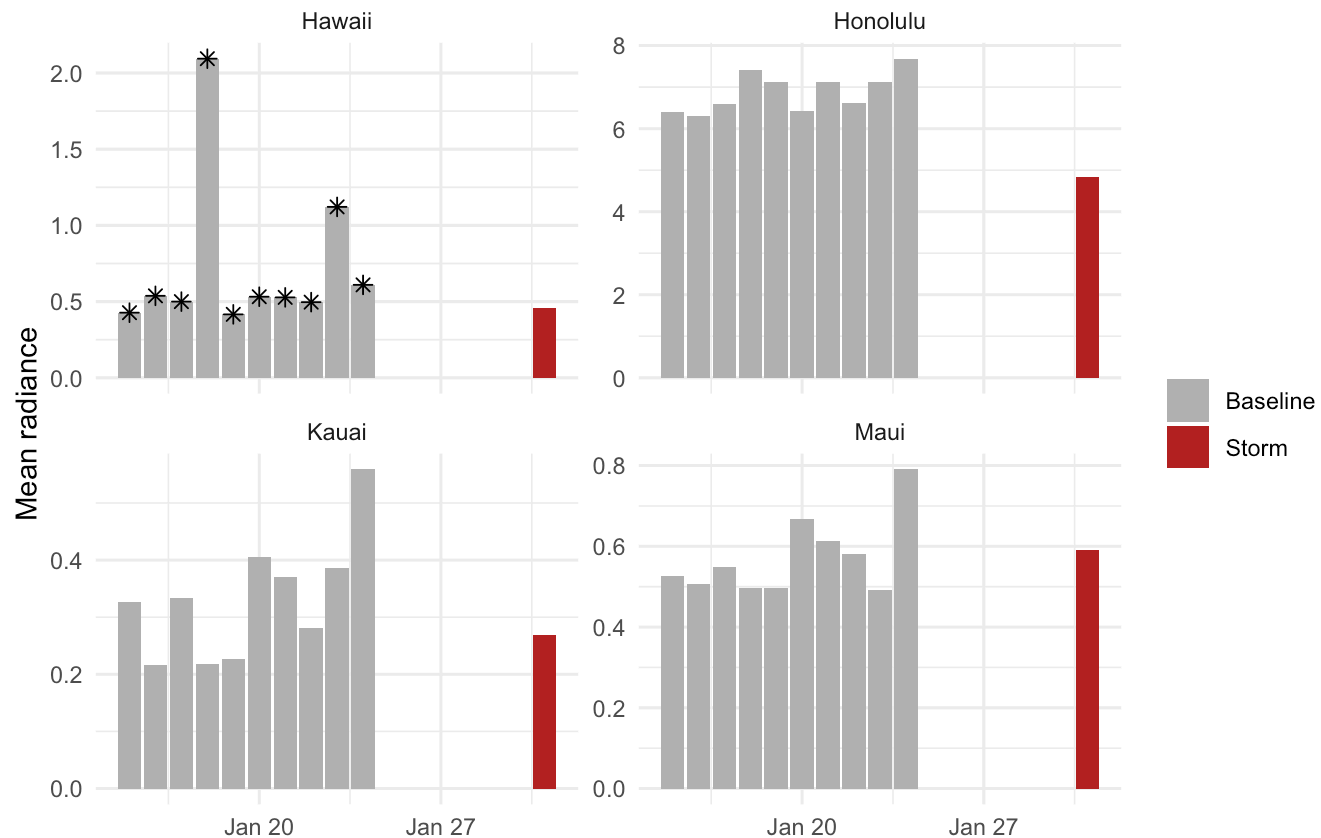
```

geom_col() +
geom_point(
  data = ntl_trend |> filter(county == "Hawaii", date %in% kilauea_dates),
  aes(x = date, y = ntl_mean), shape = 8, size = 2, color = "black",
  inherit.aes = FALSE
) +
facet_wrap(~county, scales = "free_y") +
scale_fill_manual(values = c("Baseline" = "grey70", "Storm" = "firebrick")) +
labs(title = "Mean Nighttime Radiance by Day, by County",
      subtitle = "Asterisks on Hawaii County mark nights during Kilauea's active summit eruption",
      x = NULL, y = "Mean radiance", fill = NULL) +
theme_minimal()

```

Mean Nighttime Radiance by Day, by County

Asterisks on Hawaii County mark nights during Kilauea's active summit eruption (Episode 4, beg



Scoping down to Oahu

EAGLE-I's overpass-window numbers show Honolulu County with the largest customer impact at the actual satellite capture time. From here on, the pixel-level work is scoped to Oahu — both because that's where the strongest reported signal is, and because it avoids the Kilauea confound in Hawaii County entirely rather than needing to work around it.

```
oahu_boundary <- roi_hawaii |> filter(county == "Honolulu")

baseline_mean <- mean(r_baseline, na.rm = TRUE) |> crop(oahu_boundary) |> mask(oahu_boundary)
storm_mean <- mean(r_storm, na.rm = TRUE) |> crop(oahu_boundary) |> mask(oahu_boundary)
```

Pixel-level comparison

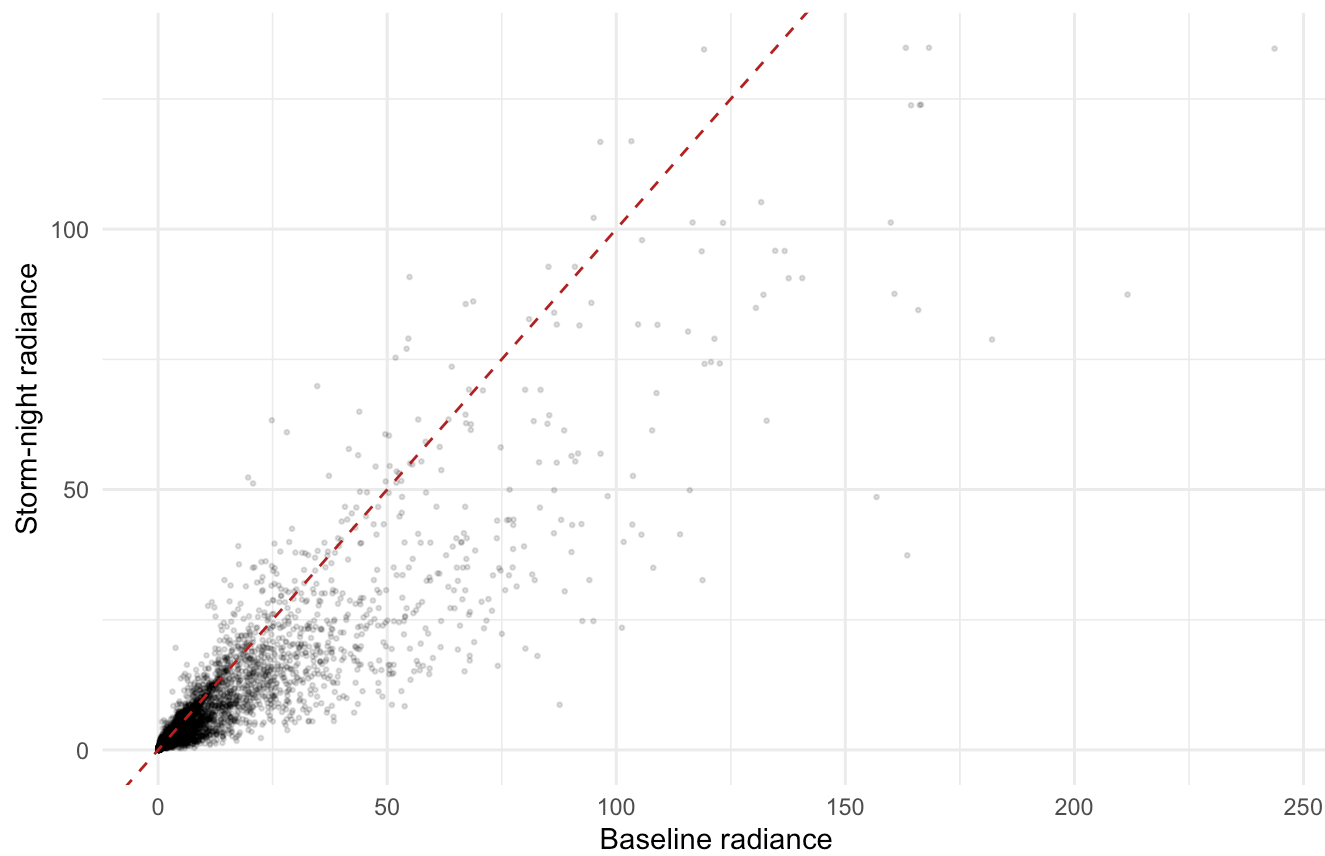
Every pixel has both a baseline value and a storm-night value. Plotting them against each other directly, pixel by pixel, is a more direct comparison than looking at either distribution separately.

```
paired_values <- tibble(
  baseline = values(baseline_mean)[, 1],
  storm = values(storm_mean)[, 1]
) |>
  filter(!is.na(baseline), !is.na(storm))

ggplot(paired_values, aes(x = baseline, y = storm)) +
  geom_point(alpha = 0.15, size = 0.5) +
  geom_abline(slope = 1, intercept = 0, color = "firebrick", linetype = "dashed") +
  labs(title = paste0("Oahu: Storm vs. Baseline Radiance, Per Pixel (", storm_date_label, ")"),
       subtitle = "Points below the dashed line got darker; above got brighter",
       x = "Baseline radiance", y = "Storm-night radiance") +
  theme_minimal()
```

Oahu: Storm vs. Baseline Radiance, Per Pixel (2025-01-31)

Points below the dashed line got darker; above got brighter



```
paired_values |>
  summarize(
    n_darkened = sum(storm < baseline),
    n_total = n(),
    pct_darkened = round(mean(storm < baseline) * 100, 1)
  )
```

A tibble: 1 × 3

n_darkened	n_total	pct_darkened
------------	---------	--------------

	<int>	<int>	<dbl>
1	4860	8269	58.8

Splitting pixels by how bright they normally are shows the darkening rate climbing steadily with brightness — close to a coin flip among the dimmest pixels, and close to universal among the brightest.

```
paired_values |>
  mutate(brightness_bin = cut(baseline, breaks = c(0, 5, 25, 75, 250),
                              labels = c("Very dim", "Dim", "Medium", "Bright"))) |>
  group_by(brightness_bin) |>
  summarize(n = n(), pct_darkened = round(mean(storm < baseline) * 100, 1))
```

A tibble: 4 × 3

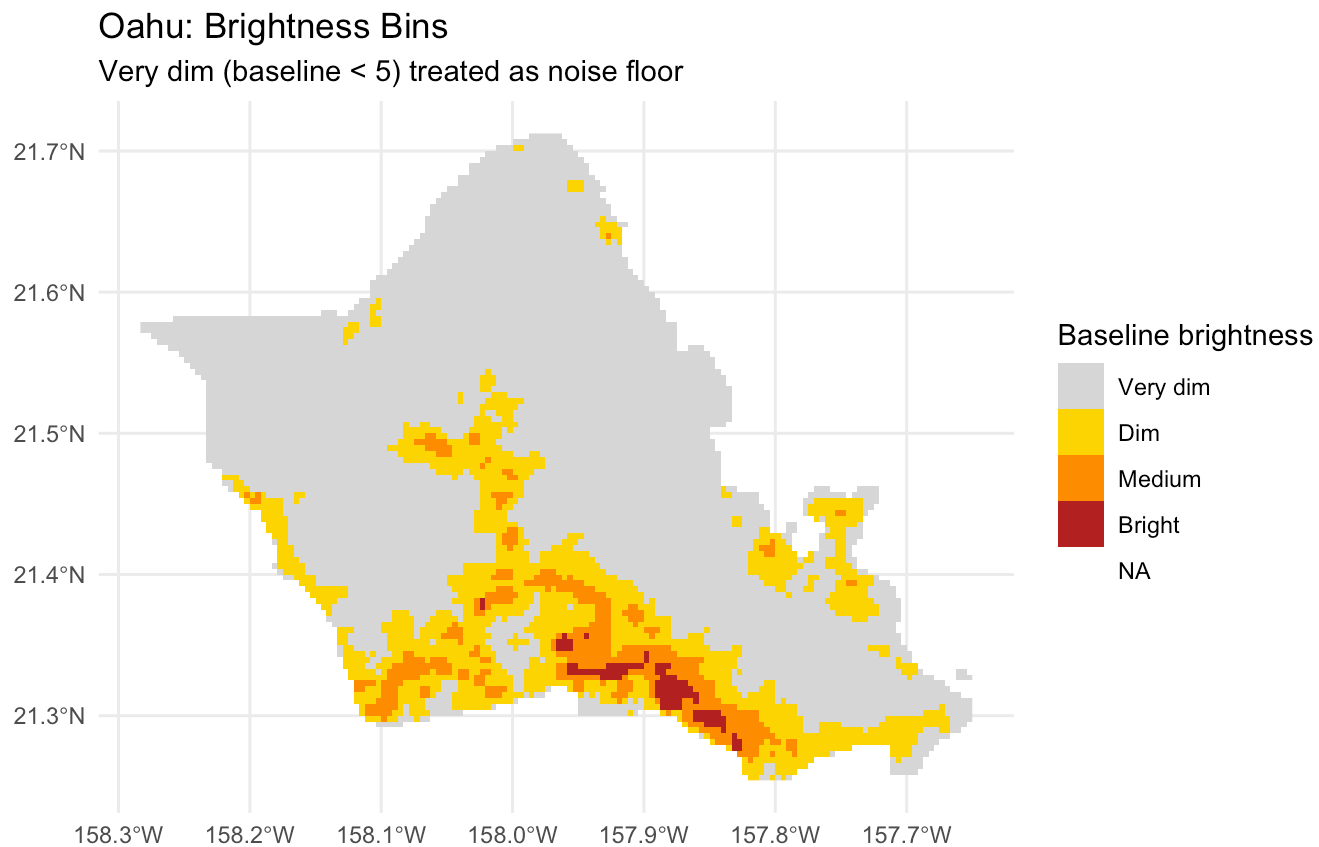
	brightness_bin	n	pct_darkened
	<fct>	<int>	<dbl>
1	Very dim	6049	51.8
2	Dim	1622	74
3	Medium	503	86.5
4	Bright	95	92.6

```
brightness_bin_raster <- classify(
  baseline_mean,
  rcl = matrix(c(
    0, 5, 1,
    5, 25, 2,
    25, 75, 3,
    75, Inf, 4
  ), ncol = 3, byrow = TRUE)
)

levels(brightness_bin_raster) <- data.frame(
  value = 1:4,
  label = c("Very dim", "Dim", "Medium", "Bright")
)

ggplot() +
  geom_spatraster(data = brightness_bin_raster) +
```

```
scale_fill_manual(
  values = c("Very dim" = "grey85", "Dim" = "gold", "Medium" = "darkorange", "Bright" = "firebr
  na.value = "transparent",
  name = "Baseline brightness"
) +
labs(title = "Oahu: Brightness Bins",
      subtitle = "Very dim (baseline < 5) treated as noise floor") +
coord_sf() +
theme_minimal()
```



Below a baseline of 5, the darkening rate is close enough to 50% that it's best treated as noise rather than signal. That threshold is used from here on to exclude pixels with no real light to lose in the first place.

```

meaningful_mask <- baseline_mean >= 5

baseline_meaningful <- mask(baseline_mean, meaningful_mask, maskvalues = FALSE)
storm_meaningful <- mask(storm_mean, meaningful_mask, maskvalues = FALSE)

abs_change_meaningful <- storm_meaningful - baseline_meaningful
print(minmax(abs_change_meaningful))

```

```

      mean
min -126.14846
max  38.48233

```

```

pct_change_meaningful <- (storm_meaningful - baseline_meaningful) / baseline_meaningful * 100
print(minmax(pct_change_meaningful))

```

```

      mean
min -94.99781
max 166.20802

```

Accounting for how much each pixel normally varies

Absolute and percent change both treat every pixel's baseline as a fixed, single number. In reality some pixels are naturally stable night to night, and others swing quite a bit even without anything unusual happening. A z-score accounts for that by scaling the change against each pixel's own typical variability.

```

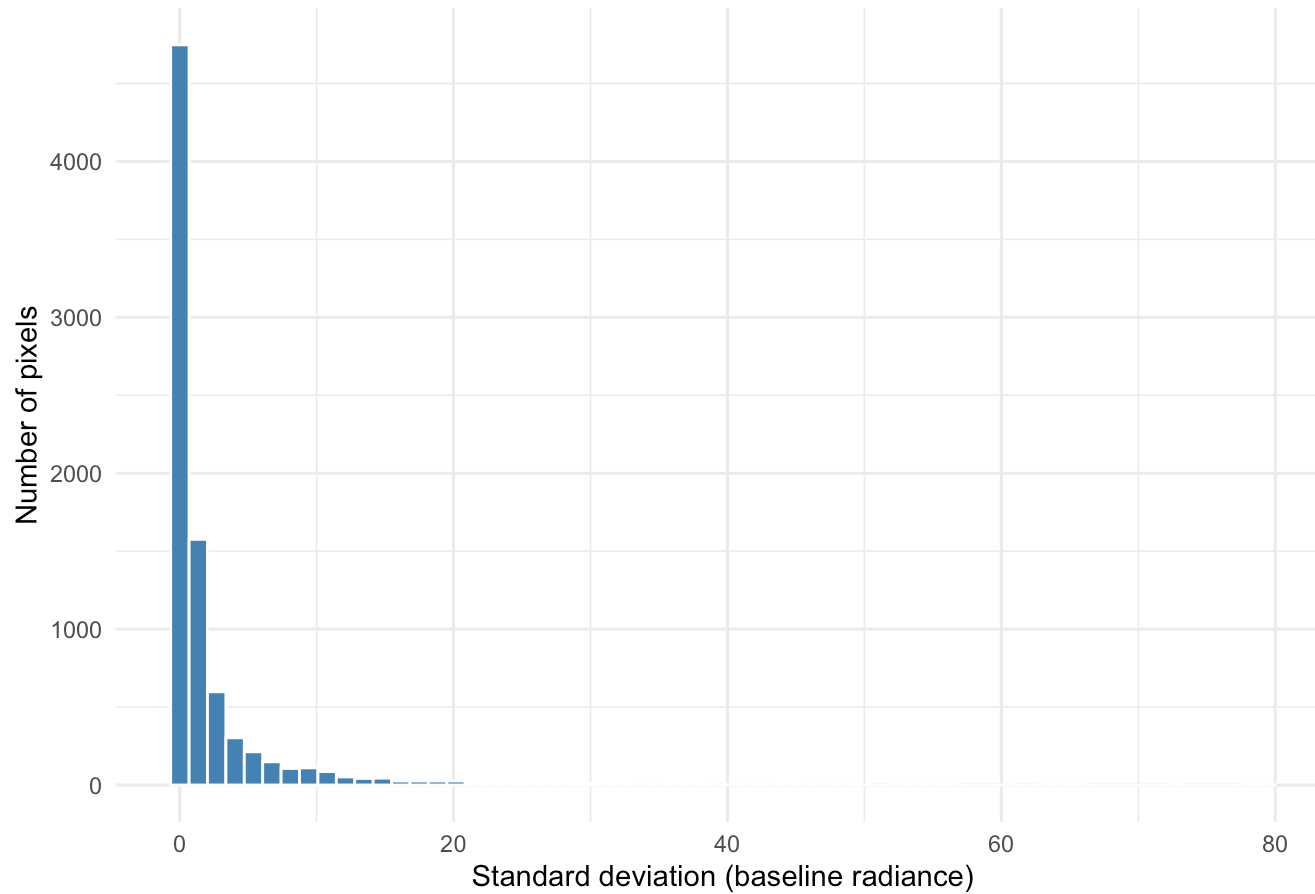
r_baseline_oahu <- crop(r_baseline, oahu_boundary) |> mask(oahu_boundary)
baseline_sd_oahu <- app(r_baseline_oahu, sd, na.rm = TRUE)

sd_values <- tibble(sd = values(baseline_sd_oahu)[, 1]) |> filter(!is.na(sd))

ggplot(sd_values, aes(x = sd)) +
  geom_histogram(bins = 60, fill = "steelblue", color = "white") +
  labs(title = "Oahu: Distribution of Per-Pixel Baseline Variability",
       x = "Standard deviation (baseline radiance)", y = "Number of pixels") +
  theme_minimal()

```

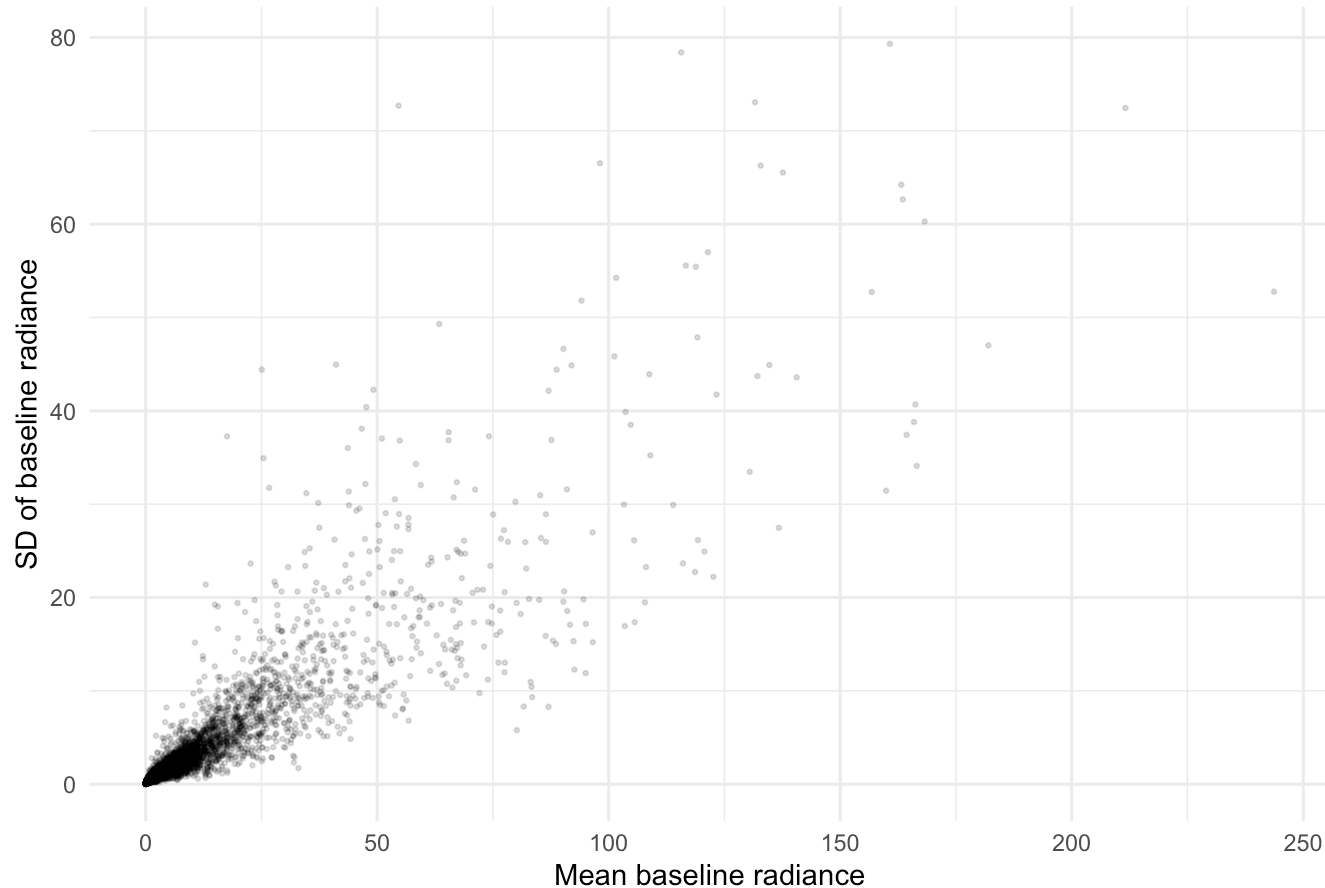
Oahu: Distribution of Per-Pixel Baseline Variability



```
sd_vs_mean <- tibble(  
  mean = values(baseline_mean)[, 1],  
  sd = values(baseline_sd_oahu)[, 1]  
) |>  
  filter(!is.na(mean), !is.na(sd))  
  
ggplot(sd_vs_mean, aes(x = mean, y = sd)) +  
  geom_point(alpha = 0.15, size = 0.5) +  
  labs(title = "Oahu: Baseline Mean vs. Baseline Variability, Per Pixel",
```

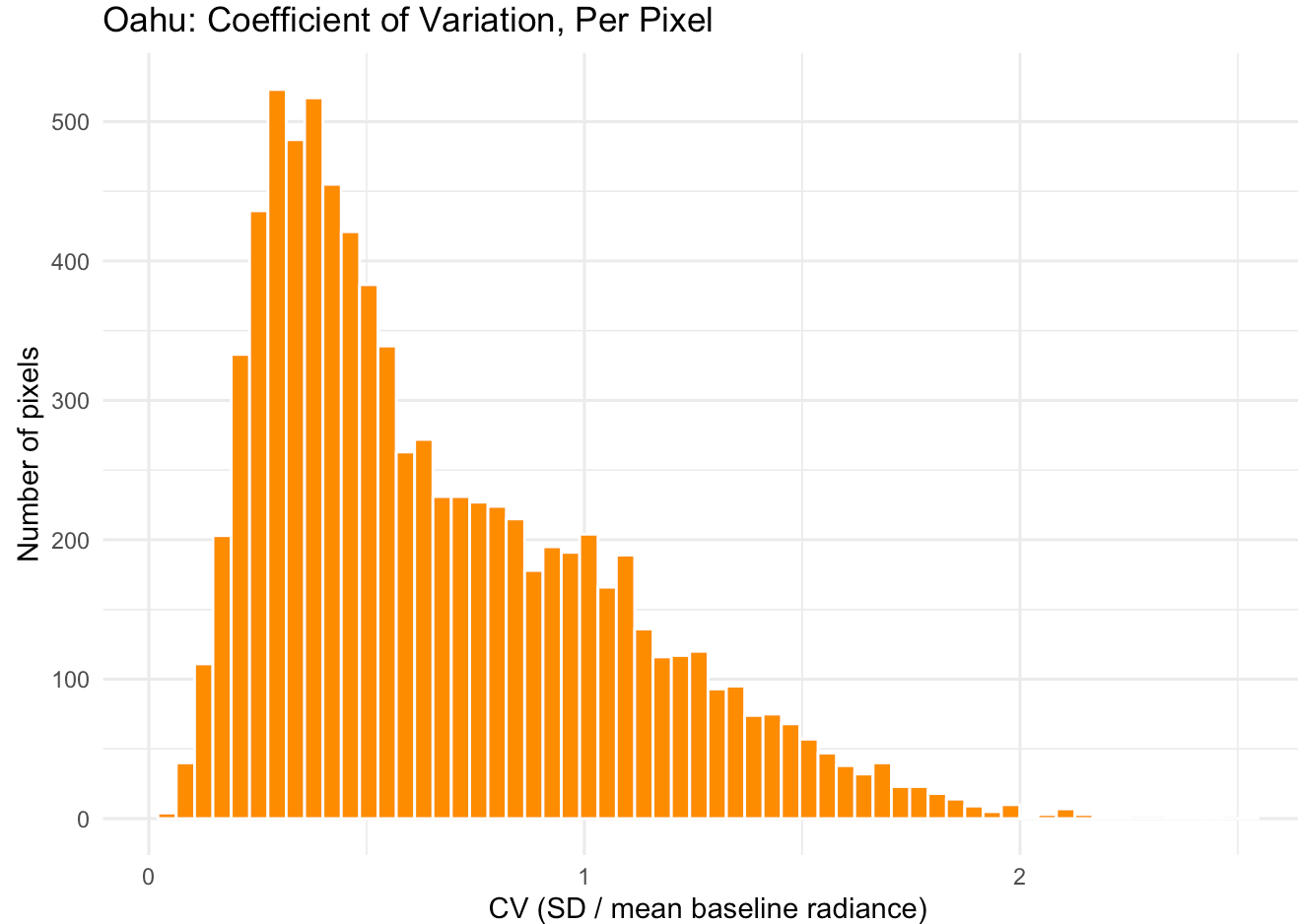
```
x = "Mean baseline radiance", y = "SD of baseline radiance") +  
theme_minimal()
```

Oahu: Baseline Mean vs. Baseline Variability, Per Pixel



```
cv_oahu <- baseline_sd_oahu / baseline_mean  
cv_values <- tibble(cv = values(cv_oahu)[, 1]) |> filter(!is.na(cv), is.finite(cv))  
  
ggplot(cv_values, aes(x = cv)) +  
  geom_histogram(bins = 60, fill = "darkorange", color = "white") +  
  labs(title = "Oahu: Coefficient of Variation, Per Pixel",
```

```
x = "CV (SD / mean baseline radiance)", y = "Number of pixels") +  
theme_minimal()
```



```
sd_threshold_oahu <- quantile(values(baseline_sd_oahu), 0.1, na.rm = TRUE)  
baseline_sd_oahu_masked <- baseline_sd_oahu  
baseline_sd_oahu_masked[baseline_sd_oahu_masked < sd_threshold_oahu] <- NA  
  
z_score_oahu <- (storm_mean - baseline_mean) / baseline_sd_oahu_masked  
print(minmax(z_score_oahu))
```

```
mean
min -10.48731
max 13.94649
```

```
z_score_meaningful <- mask(z_score_oahu, meaningful_mask, maskvalues = FALSE)
print(minmax(z_score_meaningful))
```

```
mean
min -10.487308
max 5.630037
```

Comparing the metrics

Absolute change, percent change, and z-score each answer a slightly different question, and each has situations where it's the wrong tool to reach for.

Comparing Pixel-Level Change Metrics

Nighttime radiance, baseline vs. storm night¹

Metric	Formula	Question it answers	Strengths	Weaknesses	Worked example
Absolute change	$storm - baseline$	How much raw light was lost at this exact spot?	Simple, intuitive units. No risk of blowing up near zero. Good for spotting the single brightest, most impactful drops (e.g., dense urban cores).	Naturally favors bright pixels — a big drop in a bright area looks the same as a small drop in a dim one, even if the dim one lost nearly all its light.	Baseline 100 → Storm 40: absolute change = -60. Baseline 8 → Storm 3: absolute change = -5. The bright pixel dominates any map or ranking, even though the dim pixel lost a similar share of its light.
Percent change	$(storm - baseline) / baseline \times 100$	What share of this pixel's normal light was lost?	Scales fairly across pixels of different brightness. A quiet neighborhood and a bright one can be	Explodes toward extreme, meaningless values when baseline is near zero — a tiny absolute change can register as thousands of	Baseline 100 → Storm 40: % change = -60%. Baseline 8 → Storm 3: % change = -62.5%. Now genuinely comparable —

¹ All examples use radiance units consistent with VNP46A2's default variable. Baseline mean/SD computed across 10 pre-storm nights; storm value from a single overpass.

Comparing Pixel-Level Change Metrics

Nighttime radiance, baseline vs. storm night¹

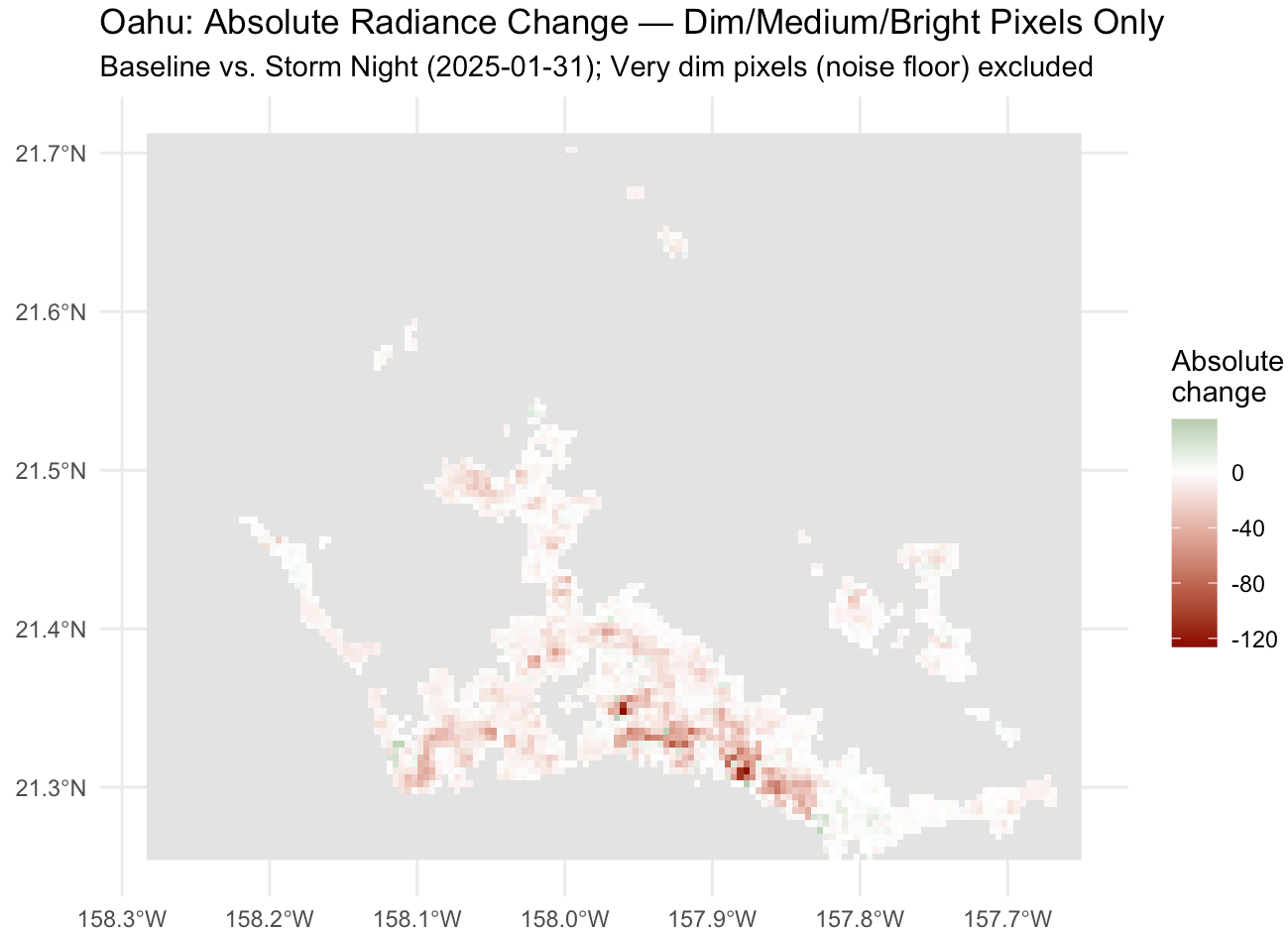
Metric	Formula	Question it answers	Strengths	Weaknesses	Worked example
			compared on the same footing.	percent. Requires masking out very dim / unpopulated pixels first.	both pixels lost roughly the same share of their light.
Z-score	$(\text{storm} - \text{baseline mean}) / \text{baseline SD}$	How unusual is this storm-night value, given how much this specific pixel normally fluctuates?	Accounts for each pixel's own natural volatility. Surfaces real change in quiet, stable areas that a small absolute or percent change might otherwise hide. Best tool for finding locally significant events.	Needs several baseline nights to estimate SD reliably. Breaks down (divides by ~zero) for perfectly flat pixels — needs the same near-zero masking as percent change. A modest z-score doesn't always mean a modest real-world impact if the pixel is naturally very volatile.	Pixel A: baseline mean 8, SD 1. Storm value 3 → $z = (3-8)/1 = -5$. A large, unusual drop. Pixel B: baseline mean 100, SD 15. Storm value 85 → $z = (85-100)/15 \approx -1$. A smaller, fairly ordinary night-to-night swing for that pixel — even though its absolute drop (-15) is three times larger than Pixel A's.
Coefficient of variation (CV)	$\text{baseline SD} / \text{baseline mean}$	How naturally volatile is this pixel, independent of how bright it is?	Distinguishes real relative instability from brightness alone. Useful diagnostic for deciding how much to trust a z-score at a given pixel.	Not itself a change metric — describes baseline behavior only, says nothing about the storm night.	Pixel with baseline mean 8, SD 1: CV = 0.13 (very stable). Pixel with baseline mean 100, SD 40: CV = 0.40 (much more erratic, relatively speaking). A z-score from the second pixel deserves more skepticism than one from the first.

¹ All examples use radiance units consistent with VNP46A2's default variable. Baseline mean/SD computed across 10 pre-storm nights; storm value from a single overpass.

Mapping the change

```
ggplot() +
  geom_spatraster(data = abs_change_meaningful) +
```

```
scale_fill_gradient2(
  low = "darkred", mid = "white", high = "darkgreen", midpoint = 0,
  na.value = "grey90", name = "Absolute\nchange"
) +
labs(title = "Oahu: Absolute Radiance Change — Dim/Medium/Bright Pixels Only",
     subtitle = paste0("Baseline vs. Storm Night (", storm_date_label, "); Very dim pixels (noise floor) excluded"),
coord_sf() +
theme_minimal()
```



```
tmap_mode("view")
```

```
i tmap modes "plot" - "view"  
i toggle with `tmap::ttm()``
```

```
tm_shape(abs_change_meaningful) +  
  tm_raster(  
    palette = "rd_yl_gn",  
    midpoint = 0,  
    title = "Absolute change",  
    style = "cont"  
  ) +  
  tm_basemap("CartoDB.DarkMatter") +  
  tm_layout(title = "Oahu: Radiance Change - Meaningful Pixels Only")
```

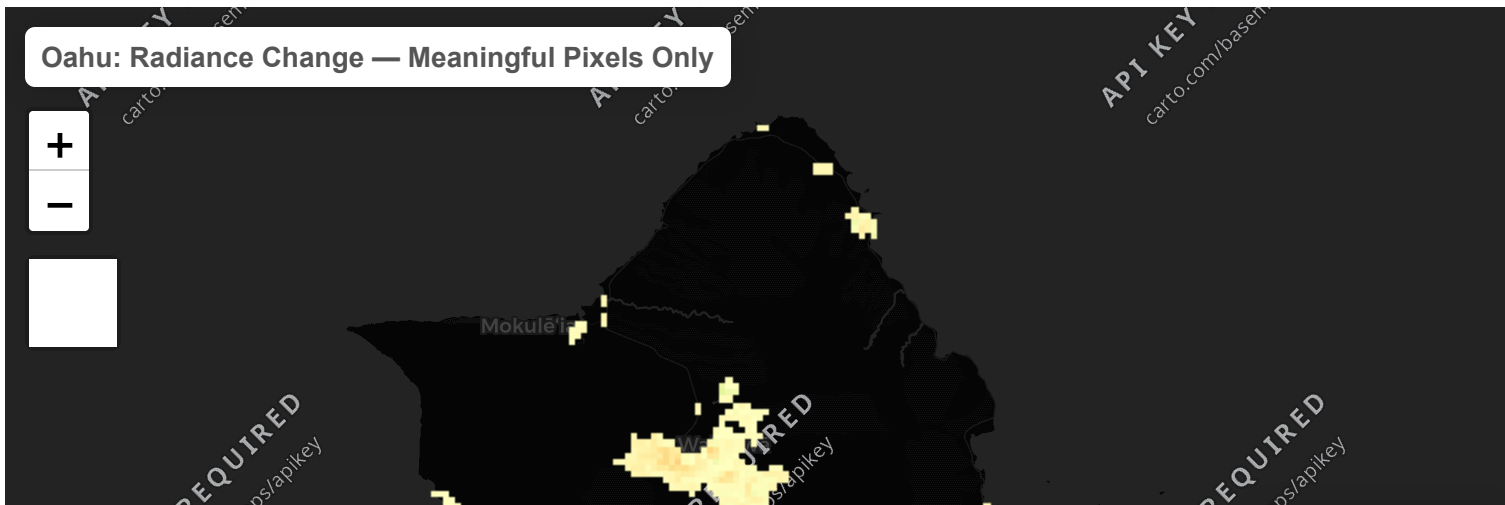
— tmap v3 code detected —

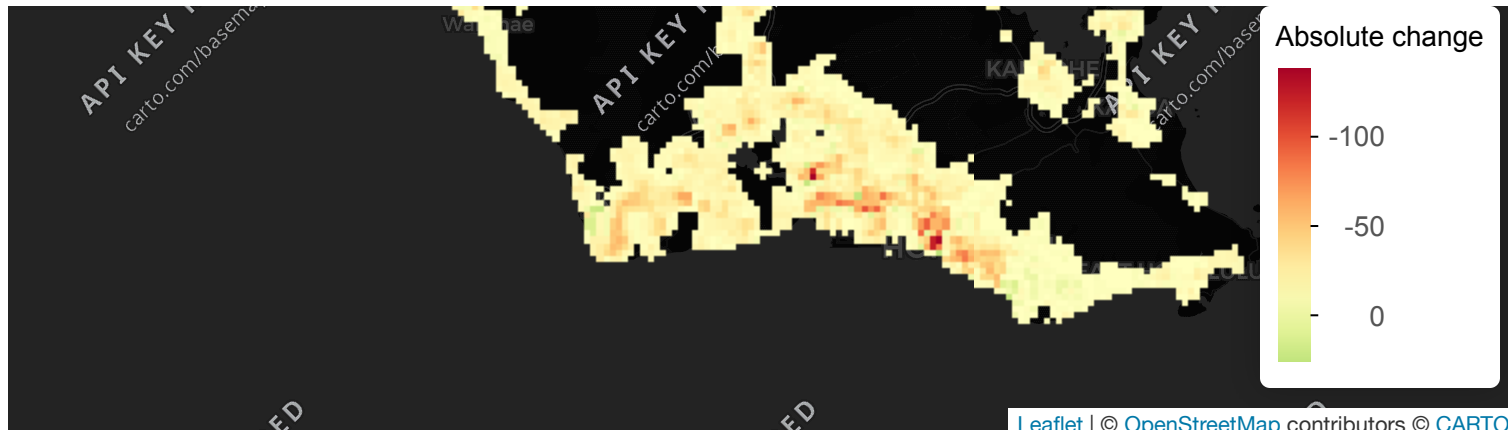
[v3->v4] `tm\_raster()`: instead of `style = "cont"`, use `col.scale =
`tm_scale_continuous()``.`

i Migrate the argument(s) 'midpoint', 'palette' (rename to 'values') to
`tm\_scale\_continuous(<HERE>)`

For small multiples, specify a 'tm_scale_' for each multiple, and put them in a
list: `'col'.scale = list(<scale1>, <scale2>, ...)` [v3->v4] `tm\_raster()`: migrate the
argument(s) related to the legend of the

map variable `col` namely 'title' to `'col.legend = tm_legend(<HERE>)` [v3->v4] `tm\_layout()`: use
`tm\_title()` instead of ``tm_layout(title = )``





```
tmap_mode("view")
```

```
i tmap modes "plot" - "view"
```

```
tm_shape(z_score_meaningful) +
  tm_raster(
    palette = "rd_yl_gn",
    midpoint = 0,
    title = "Z-score",
    style = "cont"
  ) +
  tm_basemap("CartoDB.DarkMatter") +
  tm_layout(title = "Oahu: Radiance Z-Score - Meaningful Pixels Only")
```

— tmap v3 code detected —

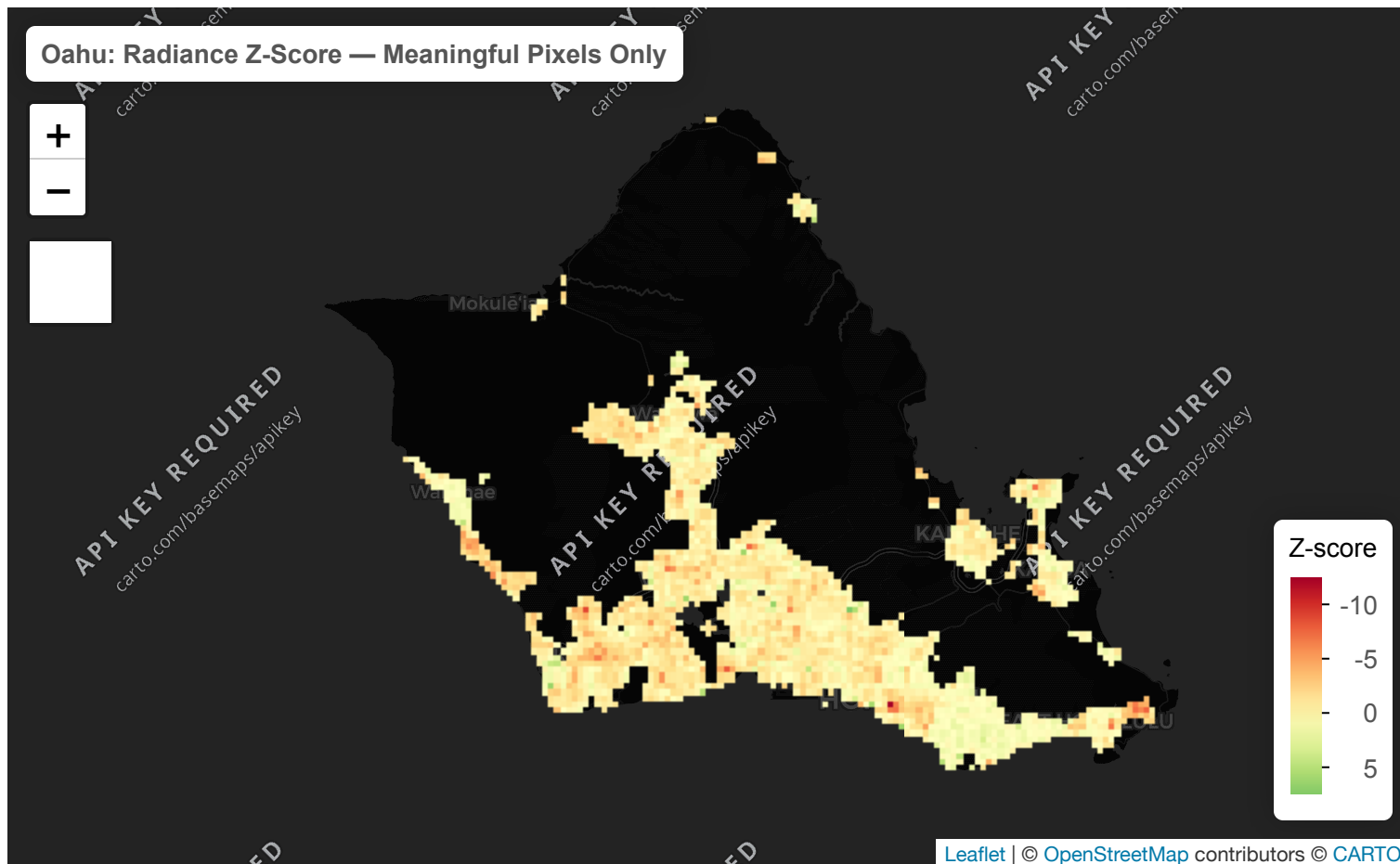
[v3->v4] `tm\_raster()`: instead of `style = "cont"`, use `col.scale =`
``tm_scale_continuous()`.`

i Migrate the argument(s) 'midpoint', 'palette' (rename to 'values') to
`'tm_scale_continuous(<HERE>)'`

For small multiples, specify a 'tm_scale_' for each multiple, and put them in a
 list: `'col'.scale = list(<scale1>, <scale2>, ...)` [v3->v4] `tm\_raster()`: migrate the

argument(s) related to the legend of the

map variable `col` namely 'title' to `'col.legend = tm_legend(<HERE>)'` [v3->v4] `tm\_layout()`: use
``tm_title()'` instead of ``tm_layout(title =)'`



Z-score by community

Aggregating the z-score up to named communities (Census-designated places) makes it possible to compare specific neighborhoods against each other directly, rather than picking individual points by hand.

```
oahu_places <- places(state = "HI", cb = TRUE, year = 2022) |>
  st_transform(4326) |>
  st_filter(oahu_boundary, .predicate = st_intersects)

place_zscores <- exact_extract(
  z_score_meaningful,
```

```
oahu_places,
  fun = "mean",
  append_cols = "NAME"
) |>
  rename(z_score = mean)
```

		0%
=		2%
==		4%
===		5%
====		7%
=====		9%
=====		11%
=====		13%
=====		15%
=====		16%
=====		18%
=====		20%
=====		22%
=====		24%
=====		25%

=====	27%
=====	29%
=====	31%
=====	33%
=====	35%
=====	36%
=====	38%
=====	40%
=====	42%
=====	44%
=====	45%
=====	47%
=====	49%
=====	51%
=====	53%
=====	55%
=====	56%
=====	58%
=====	60%

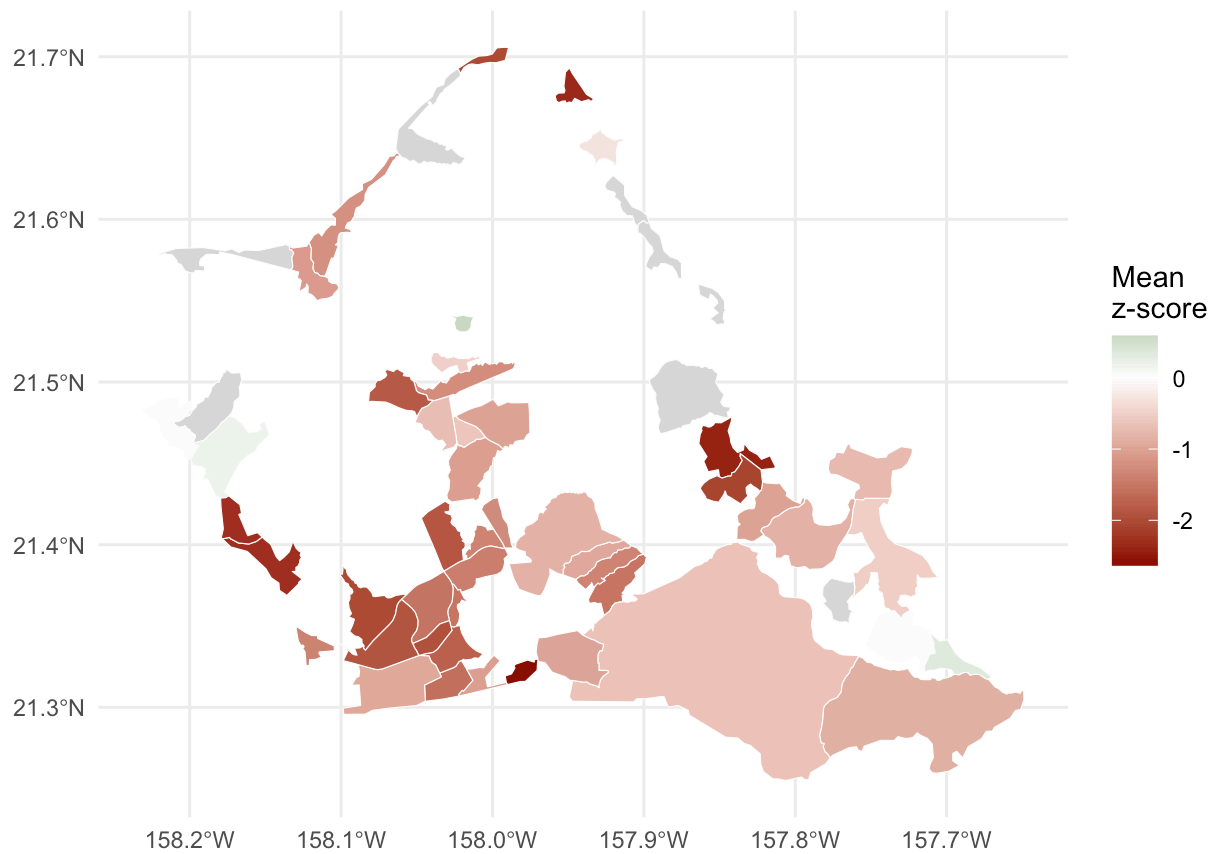
=====	62%
=====	64%
=====	65%
=====	67%
=====	69%
=====	71%
=====	73%
=====	75%
=====	76%
=====	78%
=====	80%
=====	82%
=====	84%
=====	85%
=====	87%
=====	89%
=====	91%
=====	93%
=====	95%

```
|=====| 96%  
|  
|=====| 98%  
|  
|=====| 100%
```

```
oahu_places_z <- oahu_places |>  
  left_join(place_zscores, by = "NAME")  
  
ggplot(oahu_places_z) +  
  geom_sf(aes(fill = z_score), color = "white", linewidth = 0.2) +  
  scale_fill_gradient2(  
    low = "darkred", mid = "white", high = "darkgreen", midpoint = 0,  
    na.value = "grey85", name = "Mean\nz-score"  
  ) +  
  labs(title = "Oahu: Mean Radiance Z-Score by Community (CDP)",  
        subtitle = paste0("Storm night (", storm_date_label, ") vs. baseline")) +  
  theme_minimal()
```

Oahu: Mean Radiance Z-Score by Community (CDP)

Storm night (2025-01-31) vs. baseline



```
place_zscores |>
  arrange(z_score) |>
  select(NAME, z_score)
```

	NAME	z_score
1	Iroquois Point	-2.6327190
2	Kahaluu	-2.3914452
3	Kahuku	-2.3555346
4	Nanakuli	-2.2866921
5	Maili	-2.2859051
6	Ahuimanu	-2.0703523

7	Kawela Bay	-2.0023851
8	Makakilo	-1.9842051
9	Ewa Villages	-1.9456521
10	Kapolei	-1.9088074
11	Royal Kunia	-1.8781787
12	Schofield Barracks	-1.8146212
13	Ewa Gentry	-1.7374276
14	Ocean Pointe	-1.5794713
15	East Kapolei	-1.5494729
16	West Loch Estate	-1.5430924
17	Halawa	-1.5115988
18	Waipahu	-1.4299039
19	Ko Olina	-1.3812349
20	Aiea	-1.3734945
21	Waikele	-1.3371733
22	Waipio	-1.2668726
23	Wahiawa	-1.2485710
24	Haleiwa	-1.2052640
25	Waialua	-1.0735440
26	Ewa Beach	-1.0584894
27	Mililani Town	-1.0458521
28	Heeia	-0.9859109
29	Mililani Mauka	-0.9839720
30	Hickam Housing	-0.9771098
31	Kalaeloa	-0.9517034
32	Waimalu	-0.9254733
33	East Honolulu	-0.8763256
34	Pearl City	-0.8388637
35	Kaneohe	-0.8283539
36	Kaneohe Base	-0.7311271
37	Wheeler AFB	-0.6978909
38	Urban Honolulu	-0.6410580
39	Waipio Acres	-0.6075566
40	Kailua	-0.5305811
41	Whitmore Village	-0.4939060
42	Laie	-0.2955215
43	Waimanalo	-0.0208160
44	Makaha	0.0267413

45	Waianae	0.2017954
46	Waimanalo Beach	0.3356317
47	Helemano	0.5884709
48	Hauula	NaN
49	Punaluu	NaN
50	Kaaawa	NaN
51	Pupukea	NaN
52	Makaha Valley	NaN
53	Maunawili	NaN
54	Mokuleia	NaN
55	Waiahole-Waikane	NaN